

AIRI TEAM PITB — AI INTERNSHIP

Task 2: Local Agentic AI Assistant

AI Study Buddy Agent

An Autonomous, Fully Local Agentic AI Application for
Personalized Study Planning, Quizzing, and Revision

Prepared by	Hafiza Somia Tariq
Intern Role	AI/ML Intern
Project Option	Option B — AI Study Buddy Agent
Submitted to	Omar Farooq — Associate AI/ML Engineer
Duration	1 Week

Agentic AI • Local LLMs (Ollama) • Self-Correcting Workflows • Streamlit • Tool Use

September 2026

Contents

1 Project Overview 2

2 Tools and Technologies Used 2

3 System Architecture: The 7-Step Agentic Workflow 2

4 The 7 Core Tools Catalog 3

5 Dual User Interfaces 3

5.1 Streamlit Web Application (app/main.py) 3

5.2 Interactive Terminal CLI (demo.py) 4

6 Directory Layout & File Organization 4

7 Key Features & Highlights 4

8 What I Learned 5

9 Future Improvements 5

10 Repository & Submission Links 5

1 Project Overview

The **AI Study Buddy Agent** is an autonomous, **100% local Agentic AI application** designed to help students and professionals create personalized learning plans, practice quizzes, and concise revision notes for any subject or skill level.

Unlike cloud-based tools such as ChatGPT or hosted OpenAI APIs, this project runs entirely on the local machine using **Ollama** and the **llama3.2** language model. It requires no paid API keys, no internet connection, and no cloud servers, making it completely free, private, and offline-capable.

Problem solved: Creating a coherent, time-bounded study plan — paired with a matching quiz and revision notes — is time-consuming to do manually and inconsistent in quality. This agent automates that process end-to-end, while checking and correcting its own output before returning it to the user.

Final output: A day-by-day study plan, a practice quiz with explanations, and bullet-point revision notes, generated locally, reviewed by the agent itself, and saved to disk as Markdown and JSON.

2 Tools and Technologies Used

Layer	Technology Used	Purpose
Local LLM Engine	Ollama + llama3.2:latest	Generates study plans, quizzes, summaries, and self-reviews locally.
Programming Language	Python 3.11+	Core language for agent logic, tool routing, and backend workflow.
Web UI Framework	Streamlit ($\geq 1.30.0$)	Interactive web interface with custom CSS, metric cards, and tabbed views.
HTTP Communication	requests ($\geq 2.31.0$)	Communicates with the local Ollama REST API (http://localhost:11434).
Data Persistence	JSON & Markdown (.md)	Formats and saves study plans, quizzes, and history to disk.
Path/File Management	pathlib, UTF-8	Cross-platform file path compatibility (Windows, Linux, macOS).

3 System Architecture: The 7-Step Agentic Workflow

The application does not generate text in a single shot. Instead, `app/agent.py` implements a **7-step agentic workflow** that plans, executes tools, evaluates output quality, and self-corrects before returning a result.

- 1. Input Validation** (`validate_agent_input`) — sanitizes Topic (non-empty string), Duration (1–30 days), Skill Level (`beginner/intermediate/advanced`), Daily Limit (15–480 minutes), and Format (`full, plan_and_quiz, plan_and_notes, plan_only`).
- 2. Planning Phase** — generates an explicit, numbered execution roadmap before any tool is called.

3. **Sequential Tool Execution** — automatically runs the tools required by the chosen output format.
4. **Content Review** (`content_reviewer`) — an evaluator prompt scores the draft from 1–10 and produces constructive feedback.
5. **Self-Correction Pass** — if the score is below 9/10, the agent triggers a single refinement pass to fix weak spots (e.g. missing exercises, timing errors).
6. **File & Memory Persistence** — formats Markdown output and updates historical stats in `memory.json`.
7. **Structured Output Return** — returns a clean JSON payload with metrics, tool execution logs, and file paths.

Design note: the self-correction step is what distinguishes this from a single-shot LLM call — the agent evaluates its own draft against a quality rubric and revises before the user ever sees the output.

4 The 7 Core Tools Catalog

All tools live in `app/tools.py` and are wrapped with an `@log_tool_call` decorator that tracks executed tools in real time.

1. `study_planner` — creates day-by-day schedules with realistic time limits, including auto-rescaling so daily activity minutes never exceed the user’s daily limit.
2. `quiz_generator` — generates verifiable multiple-choice and short-answer questions with detailed explanations.
3. `notes_generator` — synthesizes daily bullet-point takeaways and executive summaries.
4. `content_reviewer` — evaluates content against timing guidelines and pedagogical quality, returning a 1–10 score with suggestions.
5. `file_saver` — formats and saves Markdown (`.md`) and JSON (`.json`) files into dedicated output subdirectories.
6. `progress_saver` — updates study completion progress and quiz scores in memory.
7. `memory_tool` — manages global session stats, completed topics, and historical sessions in `outputs/saved_results/memory.json`.

5 Dual User Interfaces

5.1 Streamlit Web Application (`app/main.py`)

- **Interactive sidebar** — Topic input, duration slider, skill-level selector, daily time limit, and output format.
- **Live progress expander** — real-time step status and execution logs with tool-call badges.
- **Metric cards dashboard** — Days, Level, Daily Limit, Quality Score, Tools Executed.

- **Tabbed results view** — Study Plan, Practice Quiz, Revision Notes, Quality Review.
- **Save & download** — instant download of generated output as formatted JSON.

5.2 Interactive Terminal CLI (demo.py)

An ASCII terminal interface with input validation loops and clean, formatted terminal output — useful for headless testing and quick demos without a browser.

6 Directory Layout & File Organization

```

agentic_ai_task_2/
|-- app/
|   |-- agent.py          # 7-step core agent controller & input validation
|   |-- config.py         # Ollama health checks & model API query wrapper
|   |-- main.py           # Streamlit Web UI Application
|   |-- memory.py         # Persistent session history layer (memory.json)
|   |-- prompts.py        # JSON-constrained system prompt templates
|   '-- tools.py          # 7 core tools with @log_tool_call tracking
|-- outputs/
|   |-- generated_posts/  # Markdown study plans (*_study_plan.md)
|   |-- generated_scripts/ # Markdown quizzes & notes (*_quiz_notes.md)
|   '-- saved_results/    # JSON result payloads & memory.json statistics
|-- report/
|   '-- REPORT.md         # Comprehensive technical report & architecture doc
|-- tests/
|   |-- run_tests.py      # Automated 10-topic benchmark suite
|   |-- test_cases.md     # Generated test result summary table
|   |-- test_stage2.py    # Stage 2 individual tool & memory test
|   |-- test_stage3.py    # Stage 3 agent workflow & schema test
|   |-- test_stage4.py    # Stage 4 terminal CLI test
|   '-- test_stage5.py    # Stage 5 Streamlit UI syntax test
|-- demo.py              # Interactive Terminal CLI Application
|-- README.md            # Complete user documentation & setup guide
'-- requirements.txt      # Python dependency specifications

```

7 Key Features & Highlights

1. **100% local and free** — runs entirely on Ollama and a local LLM, with no paid cloud APIs.
2. **Self-correction (agentic AI)** — automatically reviews its own work and refines weak points before delivering output.
3. **Hallucination protection** — prompt constraints require official-documentation-style citations marked “(verify before use)” and prohibit fabricated URLs or non-existent book titles.
4. **Time-cap normalization** — automatically rescales task lengths if generated content exceeds the user’s daily time limit.

5. **Comprehensive testing** — validated through 5 test stages, including an automated 10-topic benchmark suite (`tests/run_tests.py`).

8 What I Learned

- How to design a multi-step agentic workflow (plan → execute → review → self-correct → persist) rather than a single-shot prompt-response chatbot.
- How to run and query a local LLM through Ollama’s REST API without any paid cloud dependency.
- How to build tool-calling logic where the agent decides which tools to execute based on user-selected output format.
- How to implement an automated content-quality reviewer that scores its own output and triggers a refinement pass.
- How to persist structured session memory (topics completed, scores, history) across runs using JSON.
- How to design and validate input constraints (duration, skill level, daily time budget) to keep generated plans realistic.
- The importance of systematic testing (unit, workflow, and UI-level) when building an agent with several interacting tools.

9 Future Improvements

- Swap in larger or task-specialized local models (e.g. a fine-tuned Llama variant) to improve quiz quality on advanced topics.
- Add citation verification against real, fetched documentation rather than relying solely on prompt-level constraints.
- Support multi-session, multi-topic study tracks with cross-topic progress dashboards.
- Add spaced-repetition scheduling on top of the existing study planner.
- Package the app as a one-command installer/Docker image for easier distribution.
- Deploy a public-facing demo (with a hosted-safe subset of features) for portfolio accessibility.

10 Repository & Submission Links

GitHub Repository	https://github.com/somiatariq23-dev	
Technical Report	report/REPORT.md	
User Documentation	README.md	
Saved Outputs	outputs/saved_results/, outputs/generated_scripts/	outputs/generated_posts/

— *End of Report* —